Lecture 11: Function calls

Lui Holder-Pearson

October 1, 2026

Learning Goals

- Loosely understand the memory layout of the ATMega32u2
- Understand what is meant by the term *program context*.
- Understand what the *stack* is and how it is manipulated, specifically:
  - Understand what `push`, `pop`, `(r)call`, and `ret` mean and do.
  - Understand what a stack frame is, and how it pertains to function calls.
  - Understand what the stack pointer is, and how it is used and manipulated.
- Understand what is meant by the term *program context*.

Further Reading

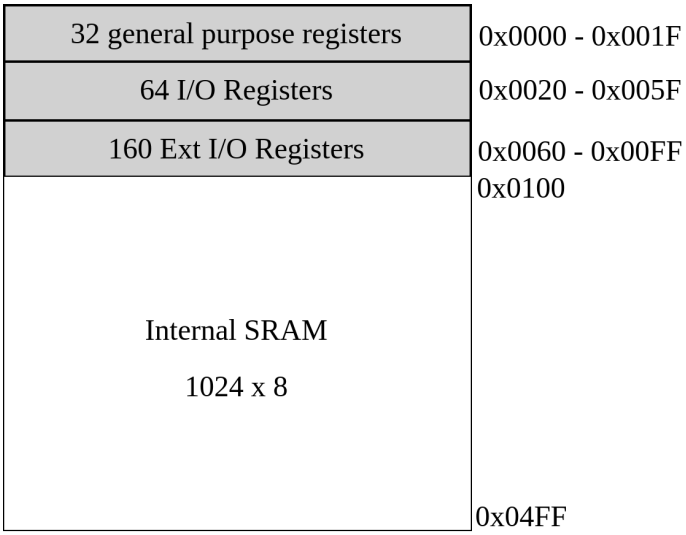
Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*,

What happens when we want to call a function?

It may be possible to simply using `brXX` instructions, but often these have a maximum modification of the PC, which may be insufficient, particularly if several functions have been declared.

1 Memory layout

The memory in the ATMega32u2 is mapped, which means that special kinds of memory and other non-memory locations can be accessed as though they are memory. The memory addressing of the ATMega32u2 is:



The I/O registers up to `0x5F` are able to be accessed through direct register addressing modes, in which the address of the register is given within the instruction. This style of addressing greatly increases the efficiency of software, as it means an address does not have to be loaded into registers first.

This is because there are 6 bits available in the instructions that directly access these addresses, and they are normalised such that A = 0x00 corresponds to global address 0x20, thus we can access:
`0b0011 1111 + 0x20`
`= 0b0101 1111 = 0x5F`

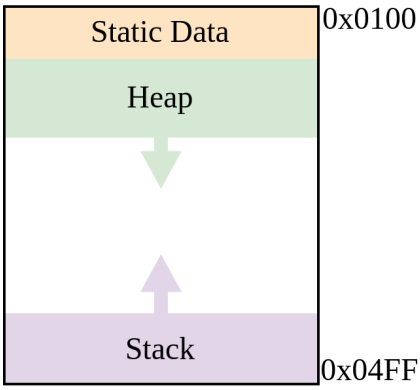
The extended I/O registers include mostly configuration registers for specific peripherals, for example the USB configuration registers, communication speed registers, and interrupt enable registers.

Syntax	Operands	Operation
`in Rd,A`	$0 \leq d \leq 31$, $0 \leq A \leq 63$	Rd $\leftarrow$ I/O(A)
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1	1011 0AAd dddd AAAA	None

1.1 The Stack

The stack refers to a memory store specific register data which contains the current context of the programme. It is stored at the opposite end of the RAM from the 'heap'. The heap is where memory that is `malloc`ed or otherwise dynamically allocated goes.

The stack begins at the highest possible memory address — `0x04FF` on the ATMega32u2 — and will decrement with continued use. The heap will begin at the lowest possible address and grow towards higher addresses.



In the `SREG` there is another special pointer similar to the `PC`, but instead pointing to the first unused address on the stack: the stack pointer — `SP`.

This special pointer readily enables storing data on the stack, with the use of the pop and the push instructions:

Syntax	Operands	Operation
`pop Rd`	$0 \leq d \leq 31$	Rd ← STACK
Control registers	Opcode	Flags
PC ← PC + 1 SP ← SP + 1	1001 000d dddd 1111	None

Syntax	Operands	Operation
`push Rr`	$0 \leq r \leq 31$	STACK ← Rr
Control registers	Opcode	Flags
PC ← PC + 1 SP ← SP - 1	1001 001d dddd 1111	None

You're probably thinking what I was thinking - if the argument is `Rr`, then why are the 'd's instead of 'r's in the opcode!?

I have no idea, but I have asked Microchip if it's a typo or not, as this is from the official documentation.

1.2 Stack frames

The stack is used for temporary storage of variables that will always be removed at some point. This is exactly the desired use when we call a function, with the `call` instruction, for example:

Syntax	Operands	Operation
`call k`	$0 \leq k \leq 64K$	STACK ← PC + 2
Control registers	Opcode	Flags
PC ← k SP ← SP - 2	1001 010k kkkk 111k kkkk kkkk kkkk kkkk	None

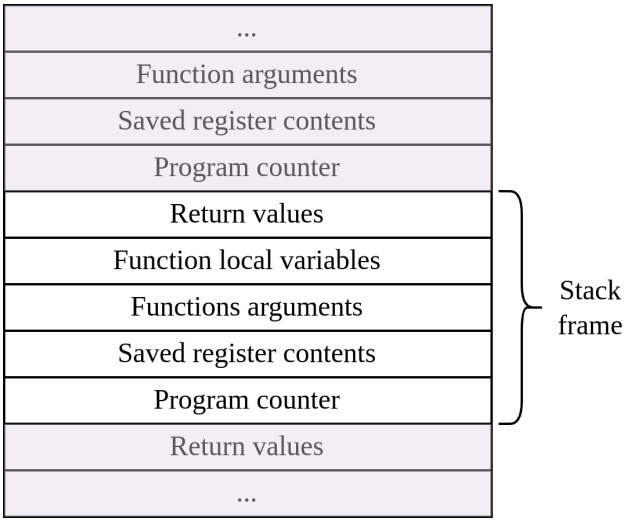
The `call` instruction is one of the notable exceptions of the 16-bit instruction.

Note: k is 22-bit to accommodate hardware with a 22-bit PC. For the ATMega32u2, k can only be 16-bit.

A stack frame is used to store the *program context*. The program context is all of the information required to complete a current function. The exact information that is stored, and the format, is defined by the compiler. Generally, information required is:

1. Return values
2. Function local variables
3. Function arguments
4. Saved register contents
5. Program counter

The programme context is stored on the stack in a *stack frame*:



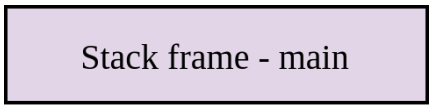
The program context is stored at the start of a function call.

```
void functionA(...) {
  // Do some stuff
}

void functionB(...) {
  ...
  functionA();
}

void functionC(...) {
  ...
  functionB();
}

int main(void) {
  ...
  functionC();
}
```



Syntax	Operands	Operation
`ret`	None	PC ← STACK
Control registers	Opcode	Flags
SP ← SP + 2	1001 0101 0000 1000	None

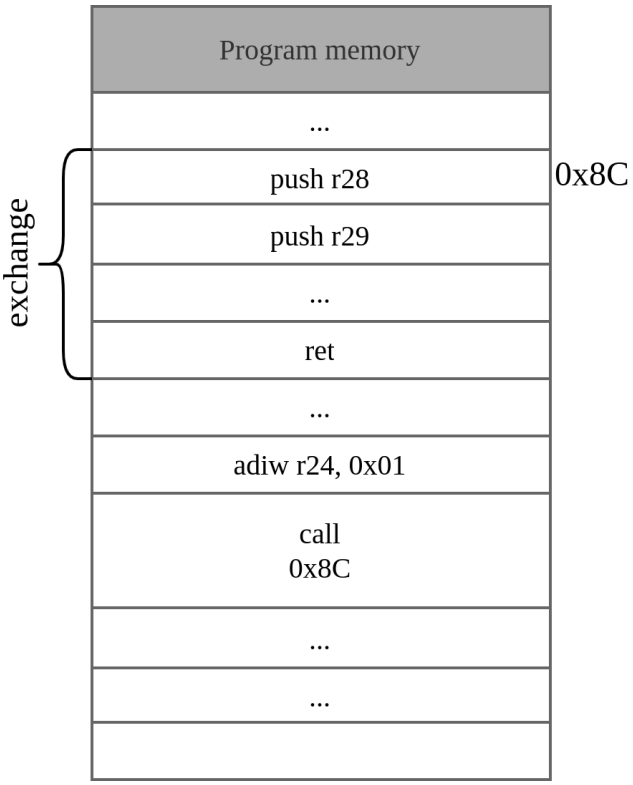
1.3 Example

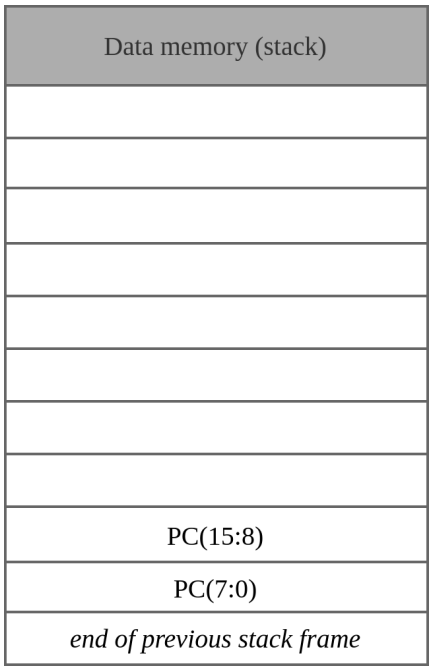
```c
#include <stdint.h>

void exchange(uint8_t *a, uint8_t *b) {
  uint8_t tmp;
  tmp = *a;
  *a = *b;
  *b = tmp;
}

int main(void) {
  uint8_t ball = 33;
  uint8_t bat = 15;
  exchange(&ball, &bat);
  return 0;
}
```

```
                        main:
e8:  ce 01        movw r24, r28
ea:  02 96        adiw r24, 0x02 ; 2
ec:  bc 01        movw r22, r24
ee:  ce 01        movw r24, r28
f0:  01 96        adiw r24, 0x01 ; 1
f2:  0e 94 46 00  call 0x8c ; exchange
```






```
                    exchange:
8c:  cf 93   push r28          
8e:  df 93   push r29          
90:  00 d0   rcall .+0         
92:  00 d0   rcall .+0         
94:  1f 92   push r1           
96:  cd b7   in r28, SPH       
98:  de b7   in r29, SPL       
9a:  9b 83   std Y+3, r25      0x03
9c:  8a 83   std Y+2, r24      0x02
9e:  7d 83   std Y+5, r23      0x05
a0:  6c 83   std Y+4, r22      0x04
                    ...
 c6  0f 90   pop r0            tmp
 c8  0f 90   pop r0            2nd arg
 ca  0f 90   pop r0            
 cc  0f 90   pop r0            1st arg
 ce  0f 90   pop r0            
 d0  df 91   pop r29           prev.  stack
 d2  cf 91   pop r28           frame
 d4  08 95   ret               
```

Summary

- Loosely understand the memory layout of the ATMega32u2
  → The ATMega32u2 is memory mapped, with the general registers, the I/O registers, extended I/O registers, and finally the SRAM being mapped into a single addressing system.
- Understand what is meant by the term program *context*.
  → The program context is all the required information to complete the current function.
- Understand what the *stack* is and how it is manipulated:
  → The stack is the section of RAM with the highest addresses. It is used entirely in a last-in, first-out manner: Only the data most recently written to the stack is accessible.

  Specifically:

  - Understand what `push`, `pop`, `(r)call`, and `ret` mean and do.
    → `push Rr` stores the value of register r on the stack.
    → `pop Rd` retrieves the current value at the top of the stack into register Rd.
    → `call k` calls a subroutine at location k in the program memory.
    → `ret` returns from the current subroutine, retrieving the address of the next instruction from the stack.
  - Understand what the stack pointer is, and how it is used and manipulated.
    → The stack pointer is the pointer to the current address of the first free space at the top of the stack. It is manipulated implicitly through the use of the `push`, `pop`, `call`, and `ret` operations.
  - Understand what a stack frame is, and how it pertains to function calls.
    → A stack frame stores the program context. Specifically, it will store:
    * The value of the program counter on return from the function.
    * Register contents before the function call
    * Function arguments.
    * Function local variables.
    * Variables returned by the function.

Next Lecture: None! Just recap

2 Questions?